

Implémentation de l'algorithme Word Count avec MapReduce sur un dataset Corpus Project Gutenberg (taille > 1 Go).

Professeur : Dr. John AOGA

Etudiant : Brice GOUDALO

Filière : Génie Logiciel

Module : Big Data

Etablissement : IFRI

Objectif du travail

- Comprendre en profondeur le paradigme MapReduce et ses phases d'exécution.
- Implémenter un algorithme Word Count complet (Mapper, Reducer, Driver) en Python via PySpark
- Gérer un dataset réaliste de plus de 1 Go issu du Project Gutenberg stocké sur HDFS.

Le paradigme MapReduce

MapReduce repose sur un principe fondamental : diviser pour régner. Un problème trop grand pour être résolu sur une seule machine est découpé en sous-problèmes indépendants (phase Map), dont les résultats intermédiaires sont agrégés (phase Reduce). Cette approche permet une parallélisation massive et transparente pour le développeur.

Le modèle de programmation expose deux fonctions pures à implémenter par le développeur :

- `Map(k1,v1)` donne `list(k2, v2)` : transforme une paire clé-valeur d'entrée en un ensemble de paires clé-valeur intermédiaires.
- `Reduce(k2, list(v2))` donne `list(v3)` : agrège toutes les valeurs associées à une même clé intermédiaire pour produire le résultat final.

Hadoop se charge intégralement de la distribution des données, de l'ordonnancement des tâches, de la tolérance aux pannes et du transfert réseau entre les nœuds.

Les Phases du Pipeline MapReduce

Phase Map

Chaque tâche Map reçoit un split (fragment) du fichier d'entrée. Par défaut, un split correspond à un bloc HDFS (128 Mo). Le Mapper lit ligne par ligne et émet des paires (clé, valeur) intermédiaires. Dans le cas du Word Count, il émet (mot, 1) pour chaque mot rencontré. Les tâches Map s'exécutent en parallèle, idéalement sur le nœud qui héberge physiquement le bloc de données concerné (localité des données).

```

#
# ÉTAPE 3: PHASE MAP - TRANSFORMATION LIGNES → TICKETS (mot, 1)
#
print("\n ÉTAPE 3: PHASE MAP - TRANSFORMATION")
print("-" * 80)

def map_function(line):
    """
    FONCTION MAP - CŒUR DE LA TRANSFORMATION

    Input:  Une ligne de texte brute du dataset
    Output: Liste de paires (mot, 1) appelées "tickets"
    """
    # Étape 1: Conversion en minuscules et extraction des mots
    # Regex \b[a-z]+\b capture uniquement les mots de lettres
    words = re.findall(r'\b[a-z]+\b', line.lower())

    # Étape 2: Création des tickets (mot, 1)
    # On ignore les mots de moins de 3 caractères pour réduire le bruit
    tickets = []
    for word in words:
        if len(word) > 2: # Filtrage des mots trop courts
            tickets.append((word, 1))

    return tickets

# Application de la phase MAP sur toutes les partitions
# flatMap étend chaque ligne en 0, 1 ou plusieurs tickets
print("Application de la transformation MAP sur toutes les partitions...")
tickets = lines.flatMap(map_function)

# Comptage et affichage des statistiques
total_tickets = tickets.count()
print(f"Tickets générés: {total_tickets:,}")
print(f"Moyenne: {total_tickets / total_lines:.1f} tickets par ligne")

```

Phase Reduce

Chaque tâche Reduce reçoit une clé et la liste de toutes les valeurs associées à cette clé, issues de l'ensemble des Mappers. Le Reducer agrège ces valeurs pour produire le résultat final. Pour le Word Count, il effectue la somme de tous les 1 pour chaque mot, produisant (mot, occurrences_totales).

```

#
# ÉTAPE 5: PHASE REDUCE - ADDITION DES COMPTEURS
#
print("\nÉTAPE 5: PHASE REDUCE - ADDITION DES COMPTEURS")
print("-" * 80)

# Application de la phase REDUCE avec reduceByKey (OPTIMAL)
print("Application de reduceByKey (optimisation réseau)...")
word_counts = tickets.reduceByKey(lambda a, b: a + b)

# Statistiques des résultats
unique_words = word_counts.count()
print(f"Mots uniques trouvés: {unique_words:,}")
print(f"Ratio unicité: {unique_words / total_tickets * 100:.2f}%")

```

Phase Shuffle & Sort

C'est la phase la plus coûteuse en termes de réseau et de disque. Elle est entièrement gérée par Hadoop sans intervention du développeur. Les paires intermédiaires émises par tous les Mappers sont triées par clé et regroupées, puis transférées vers les Reducers appropriés. Chaque Reducer reçoit l'ensemble des valeurs associées à ses clés. Un Combiner (mini-reducer local) peut être interposé pour réduire le volume de données transférées sur le réseau.

```
#
# ÉTAPE 6: SHUFFLE ET PHASE TRI - ORGANISATION PAR FRÉQUENCE
#
print("\nÉTAPE 6: SHUFFLE ET PHASE TRI - ORGANISATION PAR FRÉQUENCE")
print("-" * 80)
print("Tri des mots par fréquence décroissante")
print("Processus: inversion (count, word) → tri → réinversion")

# Étape 1: Inversion pour tri par compteur (count devient la clé)
# Exemple: ("the", 1000) → (1000, "the")
inverted = word_counts.map(lambda x: (x[1], x[0]))

# Étape 2: Tri par compteur décroissant
sorted_inverted = inverted.sortByKey(ascending=False)

# Étape 3: Réinversion pour format final (word, count)
sorted_words = sorted_inverted.map(lambda x: (x[1], x[0]))

# Affichage des 10 premiers mots (aperçu pour le rapport)
print("\nAPERÇU - TOP 10 MOTS LES PLUS FRÉQUENTS:")
print("-" * 50)
top_preview = sorted_words.take(10)

for i, (word, count) in enumerate(top_preview, 1):
    # Formatage pour affichage clair dans le rapport
    percentage = (count / total_tickets) * 100
    print(f"{i:2d}. {word:<15} → {count:>10,} occurrences ({percentage:.2f}%)"
```

Avantages et Limites de MapReduce

Aspect	Avantages	Limites
Scalabilité	Scalabilité horizontale linéaire	Efficace surtout pour les traitements batch

Tolérance aux pannes	Relance automatique des tâches échouées	Surcharge mémoire si nombreuses relances
Localité des données	Déplacement du calcul vers les données	Limité si les blocs ne sont pas locaux
Réseau	Tolérant aux latences réseau élevées	Phase Shuffle intensive en bande passante
I/O Disque	Persistance entre les phases	Goulot d'étranglement majeur pour les traitements itératifs

Description de l'Algorithme Word Count

Logique Globale

L'algorithme Word Count consiste à dénombrer la fréquence d'apparition de chaque mot distinct dans un corpus textuel. C'est le "Hello World" du Big Data, car il illustre parfaitement la décomposabilité d'un problème de traitement de données massives en tâches parallèles indépendantes.

Le traitement se déroule en trois étapes logiques :

1. Tokenisation : chaque ligne de texte est découpée en mots individuels (tokens), après normalisation (minuscules, suppression de la ponctuation).
2. Émission : pour chaque token, une paire (token, 1) est émise.
3. Agrégation : toutes les valeurs correspondant au même token sont sommées pour obtenir sa fréquence totale.

Analyse de la Complexité

Soit N le nombre total de mots dans le corpus, S le nombre de splits (blocs HDFS), W le nombre de mots distincts, et R le nombre de tâches Reduce.

Métrique	Phase Map	Phase Shuffle	Phase Reduce	Total
Temps(séq.)	$O(N/S)$ par Mapper	$O(N \log N)$	$O(N/R)$	$O(N \log N)$
Temps (paral)	$O(N/S)$ en paral	$O(N \log W)$	$O(W/R)$	$O(N \log W)$
Espace	$O(N/S)$ par Mapper	$O(N)$ réseau + disque	$O(W/R)$	$O(W)$ total

Avec S Mappers parallèles, la phase Map se réduit à $O(N/S)$, ce qui représente le gain de parallélisme principal. La phase Shuffle reste le goulot d'étranglement principal car elle nécessite un tri global $O(N \log N)$ sur l'ensemble des paires intermédiaires.

Choix Technologique

Dans le cadre de ce projet, l'implémentation du Word Count distribué a été réalisée avec Apache Spark via PySpark, plutôt qu'avec Hadoop MapReduce natif ou Hadoop Streaming.

Plusieurs approches existent pour implémenter le paradigme MapReduce :

- Hadoop MapReduce (Java natif) : solution historique, robuste mais verbeuse et moins flexible.
- Hadoop Streaming : permet d'utiliser Python via stdin/stdout, mais reste limité en performance et en expressivité.
- Apache Spark (PySpark) : framework distribué moderne basé sur le calcul en mémoire.

L'approche retenue est PySpark, pour les raisons suivantes :

- Performance élevée : grâce au traitement en mémoire (in-memory computing), contrairement à Hadoop MapReduce basé sur disque.
- Simplicité de développement : API Python expressive, beaucoup plus concise que Java MapReduce.
- Abstraction du modèle MapReduce : les opérations comme `map`, `reduceByKey`, `flatMap` implémentent directement le paradigme.
- Scalabilité : exécution distribuée automatique sur de grands volumes (> 1 Go).
- Optimisations internes : gestion du shuffle, du partitionnement et de la tolérance aux pannes.

Résultats

```
=====
RÉSUMÉ DE L'EXÉCUTION MAP REDUCE
Dataset: /home/harley/Documents/big_data/data/corpus_1_5gb.txt
Taille: 1.5GB
Partitions: 49
Lignes traitées: 34,461,944
Tickets générés: 220,796,224
Mots uniques: 111,609
Fichier sortie: /home/harley/Documents/big_data/wordcount_complet
Mot le plus fréquent: the (15,107,202 fois)

NETTOYAGE DES RESSOURCES SPARK
Session Spark arrêtée proprement
WORD COUNT MAP REDUCE TERMINÉ AVEC SUCCÈS!
✅ Terminé!
(.venv) harley@harley:~/Documents/big_data$
```